A Scale-Adapted, Cyclical Functional SGHMC Sampler: Implementation Notes and Provenance of Modifications

Abstract

This note documents the functional stochastic gradient Hamiltonian Monte Carlo (fSGHMC) sampler implemented in this codebase. The algorithm follows the functional SGMCMC framework of Wu et al. [1], in which a Gaussian-process (GP) prior over *functions* replaces the usual weight-space Gaussian prior of a Bayesian neural network (BNN), and its gradient is injected into the parameter-space dynamics through a vector–Jacobian product at a random set of measurement points. The implementation departs from the reference algorithm of [1] in several ways: the base sampler is the *scale-adapted* (preconditioned) SGHMC of Springenberg et al. [3] rather than a vanilla leapfrog SGHMC; the step size follows the cyclical cosine schedule of Zhang et al. [5] with cool-phase sample harvesting and stationary-law momentum refreshment; the posterior is tempered by a temperature parameter T; the GP prior gradient is obtained from a double-precision Cholesky solve of the nugget-regularised prior Gram matrix; and a small set of numerical safeguards (gradient-norm clipping, preconditioner floors, an optional per-step displacement clamp) protect the chain during phase transitions. We describe each modification, give its origin, and present pseudocode in the style of the original paper.

1 Setting and notation

Let $f(\cdot;\mathbf{w}) : \mathcal{X} \to \mathbb{R}$ be the function computed by a neural network with parameters $\mathbf{w} \in \mathbb{R}^k$, and let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ be a dataset with likelihood $p(y \mid f(x;\mathbf{w}))$.[1] Instead of a weight-space prior $p(\mathbf{w})$, a functional prior $P_0 \sim \mathcal{GP}(m, K)$ is placed directly on f, and the target is the posterior measure over functions

$$P_{f|\mathcal{D}}(\mathrm{d}f) \;\propto\; \exp\!\big(-\Phi(f)\big)\, P_0(\mathrm{d}f), \qquad \Phi(f) = -\log p(Y_{\mathcal{D}} \mid f(X_{\mathcal{D}};\mathbf{w})), \tag{1}$$

following the function-space formulation of [1]. Since f is determined by $\mathbf{w}$, sampling proceeds in parameter space with the potential

$$U(\mathbf{w}) \;=\; -\log p(Y_{\mathcal{D}} \mid f(X_{\mathcal{D}};\mathbf{w})) \;+\; I_0\big(f(\cdot;\mathbf{w})\big), \tag{2}$$

where I_0 is the Onsager–Machlup functional of P_0 (heuristically, $-\log P_0$). The prior term is made tractable by evaluating it on a finite set of *measurement points* $X_{\mathrm{M}} = [x_1^{\mathrm{m}}, \dots, x_M^{\mathrm{m}}]^\top$, freshly drawn at every step, where the GP marginal is a multivariate Gaussian. Writing $\mathbf{f}_{\mathrm{M}} = f(X_{\mathrm{M}};\mathbf{w})$, $\mathbf{m}_{\mathrm{M}} = m(X_{\mathrm{M}})$, $\mathbf{K}_{\mathrm{MM}} = K(X_{\mathrm{M}}, X_{\mathrm{M}})$, the stochastic estimate of $\nabla_{\mathbf{w}} U$ used at each step is

$$\nabla\widetilde{U}(\mathbf{w}) \;=\; \underbrace{\frac{N}{n}\sum_{i\in\mathcal{B}} \nabla_{\mathbf{w}}\big[-\log p(y_i \mid f(x_i;\mathbf{w}))\big]}_{\text{minibatch likelihood gradient, } |\mathcal{B}|=n} \;+\; \underbrace{J_{\mathbf{w}}(X_{\mathrm{M}})^\top \big(\mathbf{K}_{\mathrm{MM}} + \tilde{\delta} I\big)^{-1}\big(\mathbf{f}_{\mathrm{M}} - \mathbf{m}_{\mathrm{M}}\big)}_{-\nabla_{\mathbf{w}} \log P_0(\mathbf{f}_{\mathrm{M}}) \,=\, J^\top \boldsymbol{\alpha}}, \tag{3}$$

[1] The implementation is agnostic to the form of the likelihood; any differentiable $\log p(y \mid f)$ that depends on the input only through the network output fits the scheme unchanged.

where $J_{\mathbf{w}}(X_{\mathrm{M}}) \in \mathbb{R}^{M\times k}$ is the Jacobian of the network outputs at the measurement points with respect to $\mathbf{w}$ and $\tilde{\delta}$ is a diagonal regulariser (Section 2.4). This is exactly the drift of fSGHMC in Eq. (9) of [1]; only the likelihood term is minibatched, and the prior-gradient term enters unscaled, at $O(1)$ relative to the full-data likelihood gradient. The Jacobian is never formed explicitly: $J^{\top}\boldsymbol{\alpha}$ is one reverse-mode vector–Jacobian product (a single backward pass with $\boldsymbol{\alpha}$ as the output cotangent), with $\boldsymbol{\alpha}$ computed numerically and detached from the autodiff graph.

2 The implemented algorithm

2.1 Base sampler: scale-adapted SGHMC

The reference algorithm of [1] (their Algorithm 2) is a vanilla SGHMC in the sense of Chen et al. [2]: identity mass matrix, hand-tuned step size and friction, and an inner leapfrog loop of m steps with the momentum resampled from $\mathcal{N}(0, M)$ at every outer iteration. The implementation replaces this with the *scale-adapted* SGHMC of Springenberg et al. [3] (the sampler underlying BOHAMIANN), which removes the sensitivity to the raw gradient scale by adapting a diagonal preconditioner during burn-in.

The sampler maintains, per parameter element and with all operations element-wise, an exponential moving average $\hat{\mathbf{g}} \approx \mathbb{E}[\nabla\widetilde{U}]$ of the gradient, an uncentred second-moment estimate $\hat{\mathbf{v}} \approx \mathbb{E}[(\nabla\widetilde{U})^2]$, and an automatic averaging window $\boldsymbol{\tau}$ whose update rule originates in the adaptive learning-rate method of Schaul et al. [4]. During the N_b burn-in steps, for stochastic gradient $\bar{\mathbf{g}}$:

$$\boldsymbol{\tau} \leftarrow \left(1 - \frac{\hat{\mathbf{g}}^2}{\hat{\mathbf{v}} + \lambda}\right)\boldsymbol{\tau} + 1, \qquad \boldsymbol{\tau} \leftarrow \max(\boldsymbol{\tau},\, 1), \tag{4}$$

$$\hat{\mathbf{g}} \leftarrow \hat{\mathbf{g}} + \boldsymbol{\tau}^{-1}(\bar{\mathbf{g}} - \hat{\mathbf{g}}), \tag{5}$$

$$\hat{\mathbf{v}} \leftarrow \hat{\mathbf{v}} + \boldsymbol{\tau}^{-1}(\bar{\mathbf{g}}^2 - \hat{\mathbf{v}}), \qquad \hat{\mathbf{v}} \leftarrow \max(\hat{\mathbf{v}},\, v_{\min}), \tag{6}$$

with a small numerical stabiliser λ. After burn-in the estimates are frozen, so the recorded samples are drawn from unchanged (valid) dynamics, as argued in [3]. The clamps in (4) and (6) are safeguards added in this codebase; see Section 2.5.

Every step then applies the preconditioned update of [3] (their Eq. 10), written in the incremental variable $\mathbf{v} = \varepsilon M^{-1}\mathbf{r}$ (momentum after the variable substitution), with preconditioner $\mathbf{a} = \left(\sqrt{\hat{\mathbf{v}}} + \lambda\right)^{-1} \approx \hat{V}^{-1/2}$, step size ε, and momentum decay c (the implementation's `mdecay`, playing the role of $\varepsilon\hat{V}^{-1/2}C$):

$$\boldsymbol{\sigma}^2 = \max\!\left(2\,\varepsilon^2 c\,\mathbf{a} \;-\; \varepsilon^4,\; 10^{-16}\right), \tag{7}$$

$$\mathbf{v} \leftarrow \mathbf{v} \;-\; \varepsilon^2\,\mathbf{a}\odot\bar{\mathbf{g}} \;-\; c\,\mathbf{v} \;+\; \boldsymbol{\sigma}\odot\boldsymbol{\xi}, \qquad \boldsymbol{\xi} \sim \mathcal{N}(0, I), \tag{8}$$

$$\mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}. \tag{9}$$

The $-\varepsilon^4$ term in (7) is the correction for the minibatch-noise estimate $\hat{B} = \frac{1}{2}\varepsilon\hat{V}$, which cancels against the squared preconditioner exactly as in [3].

Two structural differences from Algorithm 2 of [1] follow from this choice of base sampler:

1. **No inner leapfrog loop.** The chain is a single persistent trajectory in the style of [2]: one momentum/parameter update per gradient evaluation, with friction c providing the mixing that periodic momentum resampling provides in [1]. Momentum is refreshed only at cycle boundaries (Section 2.3).

2. **Diagonal preconditioning.** The mass matrix is the adapted $M^{-1} = \mathrm{diag}(\hat{V}^{-1/2})$ rather than the identity, making the effective step size per parameter scale-invariant.

2.2 Tempering

The gradient actually passed to updates (4)–(8) is

$$\bar{\mathbf{g}} \;=\; \frac{1}{T}\,\nabla\widetilde{U}(\mathbf{w}), \tag{10}$$

where T is a posterior temperature (implemented as the sampler's gradient scale N/T applied to a per-example-normalised loss). Since the injected noise in (7) is unchanged, the chain targets the tempered posterior $\propto \left[p(Y_{\mathcal{D}}|f)\,P_0\right]^{1/T}$; $T = 1$ recovers the untempered target. Tempering the posterior of a deep BNN is common practice in SG-MCMC — Zhang et al. [5] introduce a system temperature for the same sampler family, and Wenzel et al. [7] study the effect systematically ("cold posteriors", $T < 1$). Note that both the likelihood and prior terms are tempered jointly.

2.3 Cyclical step-size schedule with cool-phase harvesting

After burn-in the step size follows the cyclical cosine schedule of Zhang et al. [5] (itself adapted from warm-restart schedules in optimisation [6]). With cycle length L, post-burn-in step index $j = (t - N_b) \bmod L$, and bounds $\varepsilon_{\min}, \varepsilon_{\max}$:

$$\varepsilon_t \;=\; \varepsilon_{\min} + \tfrac{1}{2}\big(\varepsilon_{\max} - \varepsilon_{\min}\big)\Big(1 + \cos\tfrac{\pi j}{L}\Big). \tag{11}$$

Each cycle begins with large, exploratory steps and anneals toward $\varepsilon_{\min}$; samples are recorded only near the cold end. The implementation differs from [5] in three deliberate ways:

1. **Nonzero floor.** The schedule anneals to $\varepsilon_{\min} > 0$ rather than to 0, so the chain keeps sampling (with small steps) instead of degenerating to optimisation; relatedly, there is no explicit $T = 0$ "optimisation" stage inside the exploration phase of a cycle — exploration is realised purely through the large step size. A single conventional burn-in phase precedes the first cycle (during which the preconditioner of Section 2.1 adapts), rather than treating each cycle's exploration stage as its own burn-in.

2. **Cool-phase harvesting.** Let $\rho \in (0, 1]$ be the cool fraction and $S_c \geq 1$ the samples harvested per cycle. With cool-window length $\ell = \max(1, \lfloor \rho L \rfloor)$ and thinning interval $\Delta = \max(1, \lfloor \ell / S_c \rfloor)$, a sample is recorded at step j iff the distance from the cycle end, $d = (L - 1) - j$, satisfies $d \in \{0, \Delta, 2\Delta, \ldots, (S_c - 1)\Delta\}$ — i.e., S_c thinned samples spaced to terminate exactly at the coldest step of the cycle. $S_c = 1$ keeps only the coldest iterate per cycle.

3. **Stationary-law momentum refreshment.** At each cycle start ($j = 0$) the momentum variable is resampled from its stationary distribution under the preconditioned dynamics,

$$\mathbf{v} \;\sim\; \mathcal{N}\big(0,\; \varepsilon_t^2\,\mathrm{diag}(\mathbf{a})\big), \qquad \mathbf{a} = \big(\sqrt{\hat{\mathbf{v}}} + \lambda\big)^{-1}, \tag{12}$$

with $\varepsilon_t = \varepsilon_{\max}$ at the cycle start (for unpreconditioned SGHMC this reduces to $\mathbf{v} \sim \mathcal{N}(0, \varepsilon_t I)$). This is the incremental-variable analogue of the $z_t \sim \mathcal{N}(0, M)$ refreshment in Algorithm 2 of [1]: since $\mathbf{v} = \varepsilon M^{-1}\mathbf{z}$ with $M^{-1} = \mathrm{diag}(\mathbf{a})$ and $\mathbf{z} \sim \mathcal{N}(0, M)$, one gets $\mathrm{Var}(\mathbf{v}) = \varepsilon^2 M^{-1}$. Resampling (rather than zeroing) the momentum avoids dropping the chain into an atypical set that would cost $O(1/c)$ steps to re-equilibrate.

When the cyclical schedule is disabled, the sampler falls back to a constant step size with fixed-interval thinning (record every κ-th post-burn-in iterate), matching the "draw separated samples every κ epochs" protocol of [1].

2.4 The GP prior solve

The prior-gradient cotangent $\boldsymbol{\alpha} = (\mathbf{K}_{\mathrm{MM}} + \tilde{\delta} I)^{-1}(\mathbf{f}_{\mathrm{M}} - \mathbf{m}_{\mathrm{M}})$ in Eq. (3) is computed by a direct Cholesky factorisation of the $M \times M$ Gram matrix, carried out in double precision and detached from the autodiff graph (only the subsequent VJP touches the network's computation graph). The prior kernels used in practice are full rank by construction: the kernel includes an explicit nugget (observation-noise) term $\sigma_n^2\,\delta_{x,x'}$ on the diagonal, corresponding to the valid GP kernel $k_{\mathrm{eff}}(x, x') = K(x, x') + \sigma_n^2\,\delta_{x,x'}$ [8], so the factorisation is well conditioned without further regularisation ($\tilde{\delta}$ in Eq. (3) absorbs the nugget plus any optional extra jitter). One convention is worth noting: a global prior-amplitude hyperparameter multiplies the *entire* Gram matrix, nugget included ($\mathbf{K}_{\mathrm{MM}} \to a^2\mathbf{K}_{\mathrm{MM}}$), so the conditioning of the Cholesky solve is invariant to the amplitude and the prior-gradient magnitude scales exactly as $1/a^2$. If the measurement set is empty the prior term is skipped entirely and the update reduces to likelihood-only SGHMC.

2.5 Numerical safeguards

Four safeguards, none of which appear in [1, 3, 5], protect the chain — chiefly at the transition from burn-in to the first hot phase of the cyclical schedule, where the preconditioner calibrated on burn-in gradients suddenly meets $\varepsilon_{\max}$-sized steps:

- **Second-moment floor** ($v_{\min}$, Eq. 6). A parameter that receives near-zero gradient during burn-in (e.g. a dead ReLU unit) drives $\hat{\mathbf{v}} \to 0$, and with only the tiny stabiliser λ in the denominator the preconditioner gain $1/(\sqrt{\hat{\mathbf{v}}} + \lambda)$ can reach $\sim 10^8$, producing a catastrophic jump on the first nonzero gradient. Flooring $\hat{\mathbf{v}} \geq v_{\min}$ bounds the gain by $v_{\min}^{-1/2}$.
- **Averaging-window clamp** ($\boldsymbol{\tau} \geq 1$, Eq. 4). Floating-point rounding can push $\boldsymbol{\tau}$ below 1 when gradients are small; a negative $\boldsymbol{\tau}^{-1}$ would reverse the moving-average updates and can drive $\hat{\mathbf{v}}$ negative ($\sqrt{\hat{\mathbf{v}}} = \mathrm{NaN}$).
- **Gradient-norm clipping.** The total gradient norm (of the un-tempered, per-example-scale gradient) is clipped to a threshold κ_{clip} — always during burn-in, and during the sampling phase only if explicitly enabled, since clipping while samples are being recorded biases the sampled posterior (in particular its tails). The pre-clip norm is instrumented in both phases so that runs can verify the clip never fires while sampling.
- **Displacement clamp** (optional). Each element of $\mathbf{v}$ may be clamped to $[-v_{\max}, v_{\max}]$ before the position update (9), bounding the per-step parameter change. The steady-state momentum scale is $O(\varepsilon^2/c)$; the clamp is set well above that level so it fires only in pathological transients.

2.6 Initialisation and parallel chains

Rather than initialising $\mathbf{w}_0 \sim \mathcal{N}(0, I)$ as in [1], chains start from a shared warm-up point (e.g. the end state of a preliminary burn-in-only run). When R chains are run in parallel, chain r perturbs every parameter tensor of the shared start by $\mathcal{N}\big(0, (\iota \cdot \mathrm{std}(\mathbf{w}))^2\big)$ with a relative jitter

scale ι, giving overdispersed initial states so that pooled-chain convergence diagnostics such as $\widehat{R}$ remain valid and the chains can cover distinct posterior modes [9]. Chains are otherwise fully independent (separate seeds, RNG streams, and sample stores).

3 Summary of modifications and their origins

Component / modification	**Origin**
Functional prior gradient at random measurement points; parameter-space functional dynamics	Wu et al. [1] (fSGHMC)
Persistent-momentum SGHMC with friction (no leapfrog inner loop, no MH correction)	Chen et al. [2]
Diagonal preconditioner $M^{-1} = \mathrm{diag}(\hat{V}^{-1/2})$ adapted during burn-in; noise-estimate cancellation; frozen after burn-in	Springenberg et al. [3] (BOHAMIANN)
Automatic averaging window $\boldsymbol{\tau}$ for the moment estimates	Schaul et al. [4], via [3]
Cyclical cosine step size; exploration/sampling stages; samples from the cold phase	Zhang et al. [5] (cSG-MCMC); cosine restarts from [6]
Posterior temperature T	Zhang et al. [5]; see also Wenzel et al. [7]
Momentum refreshment at cycle starts from the preconditioned stationary law (12)	Refreshment per Algorithm 2 of [1]; the preconditioned form and cycle-start placement are this codebase's
Multi-sample cool-phase harvesting (S_c thinned samples ending at the coldest step)	This codebase
Double-precision Cholesky solve of the nugget-regularised Gram; amplitude-invariant conditioning convention	Standard GP practice [8]; conventions are this codebase's
Safeguards: $v_{\min}$ floor, $\boldsymbol{\tau} \geq 1$ clamp, phase-scoped gradient clipping, displacement clamp	This codebase (heuristics; not from the literature)
Warm-started, overdispersed parallel chains	Standard MCMC practice [9]

Table 1: Provenance of each component of the implemented sampler.

4 Pseudocode

Algorithm 1 gives the per-iteration update (the scale-adapted SGHMC step applied to the functional-posterior gradient), and Algorithm 2 the full sampler, in the style of Algorithm 2 of [1]. All vector operations are element-wise.

As in [1], posterior predictions integrate over the collected (pooled, over chains) samples: $p(y^*|x^*, \mathcal{D}) \approx \frac{1}{|\mathcal{S}|} \sum_{\mathbf{w}\in\mathcal{S}} p\big(y^* \mid f(x^*; \mathbf{w})\big)$.

Algorithm 1 AdaptiveStep: scale-adapted SGHMC update [3]

Require: gradient $\bar{\mathbf{g}}$; step size ε; momentum decay c; state $(\boldsymbol{\tau}, \hat{\mathbf{g}}, \hat{\mathbf{v}}, \mathbf{v})$; step counter t; burn-in length N_b; stabiliser λ; floor $v_{\min}$; optional clamp $v_{\max}$

if $t \le N_b$ **then** ▷ burn-in: adapt preconditioner
  $\boldsymbol{\tau} \leftarrow \big(1 - \hat{\mathbf{g}}^2/(\hat{\mathbf{v}} + \lambda)\big)\boldsymbol{\tau} + 1; \quad \boldsymbol{\tau} \leftarrow \max(\boldsymbol{\tau}, 1)$
  $\hat{\mathbf{g}} \leftarrow \hat{\mathbf{g}} + \boldsymbol{\tau}^{-1}(\bar{\mathbf{g}} - \hat{\mathbf{g}})$
  $\hat{\mathbf{v}} \leftarrow \hat{\mathbf{v}} + \boldsymbol{\tau}^{-1}(\bar{\mathbf{g}}^2 - \hat{\mathbf{v}}); \quad \hat{\mathbf{v}} \leftarrow \max(\hat{\mathbf{v}}, v_{\min})$
end if
$\mathbf{a} \leftarrow \big(\sqrt{\hat{\mathbf{v}}} + \lambda\big)^{-1}$ ▷ $\approx \hat{V}^{-1/2}$
$\boldsymbol{\sigma}^2 \leftarrow \max\big(2\varepsilon^2 c\,\mathbf{a} - \varepsilon^4,\, 10^{-16}\big)$
$\mathbf{v} \leftarrow \mathbf{v} - \varepsilon^2 \mathbf{a} \odot \bar{\mathbf{g}} - c\,\mathbf{v} + \boldsymbol{\sigma} \odot \boldsymbol{\xi}, \quad \boldsymbol{\xi} \sim \mathcal{N}(0, I)$
if $v_{\max}$ set **then** $\mathbf{v} \leftarrow \mathrm{clamp}(\mathbf{v}, -v_{\max}, v_{\max})$
end if
$\mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}$

References

[1] M. Wu, J. Xuan, and J. Lu. Functional stochastic gradient MCMC for Bayesian neural networks. In *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics (AISTATS)*, PMLR 258, 2025.

[2] T. Chen, E. Fox, and C. Guestrin. Stochastic gradient Hamiltonian Monte Carlo. In *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 2014.

[3] J. T. Springenberg, A. Klein, S. Falkner, and F. Hutter. Bayesian optimization with robust Bayesian neural networks. In *Advances in Neural Information Processing Systems 29 (NeurIPS)*, 2016.

[4] T. Schaul, S. Zhang, and Y. LeCun. No more pesky learning rates. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, 2013.

[5] R. Zhang, C. Li, J. Zhang, C. Chen, and A. G. Wilson. Cyclical stochastic gradient MCMC for Bayesian deep learning. In *International Conference on Learning Representations (ICLR)*, 2020.

[6] I. Loshchilov and F. Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017.

[7] F. Wenzel, K. Roth, B. S. Veeling, J. Świątkowski, L. Tran, S. Mandt, J. Snoek, T. Salimans, R. Jenatton, and S. Nowozin. How good is the Bayes posterior in deep neural networks really? In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[8] C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

[9] A. Gelman and D. B. Rubin. Inference from iterative simulation using multiple sequences. *Statistical Science*, 7(4):457–472, 1992.

Algorithm 2 Cyclical scale-adapted functional SGHMC (this codebase)

Require: dataset $\mathcal{D}$ ($|\mathcal{D}| = N$), minibatch size n; GP prior $\mathcal{GP}(m, K)$ with jitter δ; measurement pool $\mathcal{X}_{\text{pool}}$ and draw size M; temperature T; burn-in steps N_b; cycle length L; step sizes $\varepsilon_{\min}, \varepsilon_{\max}$; cool fraction ρ; samples per cycle S_c; discard count D_0; target sample count S_{tot}; momentum decay c; clip threshold κ_{clip} (burn-in only, unless enabled for sampling); warm start $\mathbf{w}^\star$, jitter scale ι

$\mathbf{w}_0 \leftarrow \mathbf{w}^\star + \iota\,\mathrm{std}(\mathbf{w}^\star) \odot \boldsymbol{\zeta}, \quad \boldsymbol{\zeta} \sim \mathcal{N}(0, I)$ ▷ overdispersed init (per chain)
$(\boldsymbol{\tau}, \hat{\mathbf{g}}, \hat{\mathbf{v}}) \leftarrow \mathbf{1}; \quad \mathbf{v} \leftarrow \mathbf{0}; \quad \mathcal{S} \leftarrow \emptyset; \quad s \leftarrow 0$
for $t = 0, 1, 2, \ldots$ until $|\mathcal{S}| = S_{\text{tot}}$ **do**
  # **Step size and momentum refreshment**
  if $t < N_b$ **then** $\quad \varepsilon_t \leftarrow \varepsilon_{\min}$
  else
    $j \leftarrow (t - N_b) \bmod L; \qquad \varepsilon_t \leftarrow \varepsilon_{\min} + \frac{1}{2}(\varepsilon_{\max} - \varepsilon_{\min})\big(1 + \cos\frac{\pi j}{L}\big)$
    if $j = 0$ **then** $\quad \mathbf{v} \sim \mathcal{N}\big(0,\ \varepsilon_t^2\,\mathrm{diag}\big((\sqrt{\hat{\mathbf{v}}} + \lambda)^{-1}\big)\big)$ ▷ Eq. (12)
    end if
  end if
  # **Stochastic functional-posterior gradient**
  draw minibatch $\mathcal{B} \subset \mathcal{D}$, $|\mathcal{B}| = n; \qquad \mathbf{g}_{\text{lik}} \leftarrow \frac{1}{n}\sum_{i \in \mathcal{B}} \nabla_{\mathbf{w}}\big[-\log p(y_i \mid f(x_i; \mathbf{w}_t))\big]$
  draw measurement set $X_{\text{M}} \subset \mathcal{X}_{\text{pool}}$ without replacement, $|X_{\text{M}}| = M$
  $\mathbf{f}_{\text{M}} \leftarrow f(X_{\text{M}}; \mathbf{w}_t); \qquad \boldsymbol{\alpha} \leftarrow (\mathbf{K}_{\text{MM}} + \tilde{\delta} I)^{-1}(\mathbf{f}_{\text{M}} - \mathbf{m}_{\text{M}})$ ▷ Cholesky, §2.4
  $\mathbf{g}_{\text{prior}} \leftarrow J_{\mathbf{w}}(X_{\text{M}})^\top \boldsymbol{\alpha}$ ▷ one VJP backward pass
  $\bar{\mathbf{g}} \leftarrow \mathbf{g}_{\text{lik}} + \mathbf{g}_{\text{prior}}/N$
  if $t < N_b$ **or** clip-in-sampling enabled **then**
    $\bar{\mathbf{g}} \leftarrow \bar{\mathbf{g}} \cdot \min\big(1,\ \kappa_{\text{clip}}/\|\bar{\mathbf{g}}\|_2\big)$
  end if
  $\bar{\mathbf{g}} \leftarrow (N/T)\,\bar{\mathbf{g}}$ ▷ $= \frac{1}{T}\nabla\widetilde{U}(\mathbf{w}_t)$, Eq. (3)
  # **Update**
  $\mathbf{w}_{t+1} \leftarrow \textsc{AdaptiveStep}(\bar{\mathbf{g}}, \varepsilon_t, c, \text{state}, t, N_b)$ ▷ Algorithm 1
  # **Cool-phase sample harvesting**
  if $t \geq N_b$ **then**
    $\ell \leftarrow \max(1, \lfloor \rho L \rfloor); \quad \Delta \leftarrow \max(1, \lfloor \ell / S_c \rfloor); \quad d \leftarrow (L - 1) - j$
    if $d \in \{0, \Delta, \ldots, (S_c - 1)\Delta\}$ **then**
      $s \leftarrow s + 1; \qquad$ **if** $s > D_0$ **then** $\mathcal{S} \leftarrow \mathcal{S} \cup \{\mathbf{w}_{t+1}\}$
    end if
  end if
end for
return $\mathcal{S}$